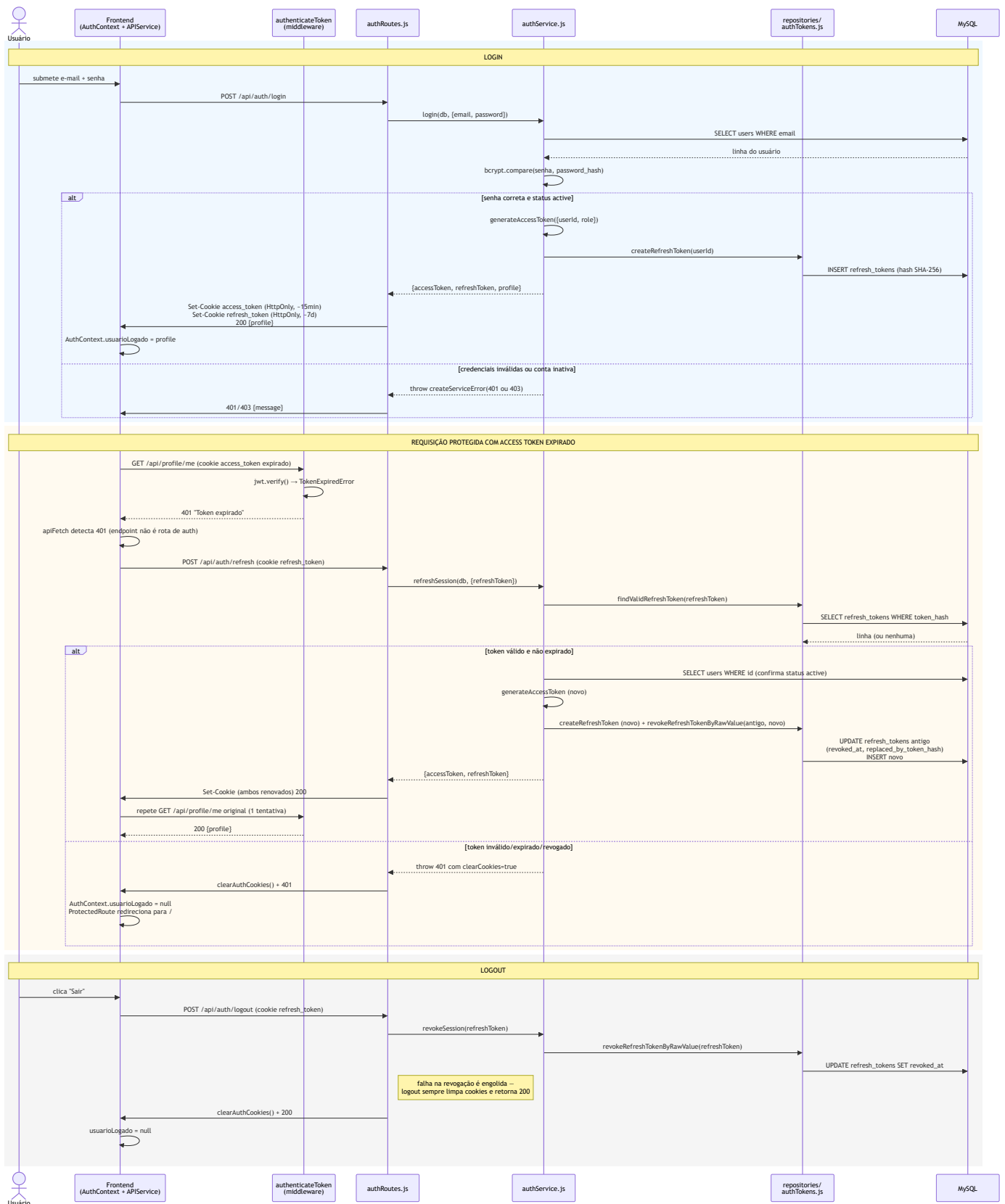


Diagrama — Fluxo de Autenticação

Cobre login, silent-refresh automático e logout, com os nomes reais de arquivos e funções (`authRoutes.js` , `authService.js` , `repositories/authTokens.js` , `APIService.jsx` , `AuthContext.jsx`).



Que problema este diagrama esclarece

Autenticação por cookie com refresh silencioso é fácil de entender mal em prosa ("o token renova sozinho") mas tem detalhes de timing e tratamento de erro que só um diagrama de sequência deixa

inequívocos: quantas tentativas de refresh acontecem, o que dispara cada uma, e o que acontece exatamente quando o refresh também falha. Este diagrama existe para que ninguém precise adivinhar esse comportamento lendo `APIService.jsx` e `authRoutes.js` ao mesmo tempo.

Como interpretar os elementos

- As três caixas coloridas (`rect`) são três cenários independentes, não uma sequência única — login, refresh-sob-401 e logout acontecem em momentos diferentes da vida da aplicação.
- O `alt / else` em cada bloco mostra o caminho de sucesso e o de falha lado a lado — ambos são comportamento real, verificado no código (`authService.js`), não uma simplificação.
- A nota "1 tentativa" no bloco de refresh é literal: `APIService.jsx` memoiza a promise de refresh entre requisições concorrentes e nunca tenta um segundo refresh para a mesma requisição original — se o refresh falhar, o erro da tentativa de refresh é propagado, não um novo retry.
- `clearCookies=true` anexado ao erro lançado por `refreshSession` é um detalhe de implementação real: o service de auth não manipula `res` diretamente (não é responsabilidade dele), então sinaliza para a rota, via uma propriedade no erro, que os cookies também precisam ser limpos — a rota é quem efetivamente chama `clearAuthCookies(res)` .

Que decisões arquiteturais este diagrama evidencia

- **Nenhum token vive em JavaScript acessível** — só em cookies `HttpOnly` . O diagrama nunca mostra o frontend lendo o valor do token, só reagindo a códigos de status.
- **Rotação de refresh token a cada uso**: o token antigo é explicitamente revogado e ligado ao novo (`replaced_by_token_hash`) no mesmo passo em que o novo é criado — não há uma janela em que dois refresh tokens válidos coexistem para a mesma sessão por design.
- **Logout é deliberadamente resiliente a falha**: a nota no diagrama ("falha na revogação é engolida") reflete uma escolha explícita no código — o usuário nunca deve ficar "preso" logado porque a revogação no banco falhou; a experiência local de logout sempre funciona.

Trade-offs que este fluxo representa

- **Memoização de refresh entre requisições concorrentes** evita uma tempestade de chamadas a `/api/auth/refresh` quando várias chamadas de API disparam 401 ao mesmo tempo (ex. uma página que faz 3 requisições paralelas ao carregar), mas significa que, se o refresh falhar, todas as 3 requisições originais falham juntas, mesmo que talvez uma delas fosse para um endpoint público que nem precisasse de autenticação.

- **Access token de vida curta (~15 min) + refresh de ~7 dias:** reduz a janela de uso de um access token roubado, mas implica que praticamente toda sessão de uso real do produto vai passar por pelo menos um refresh silencioso — o custo de latência extra desse round-trip adicional é aceito em troca da postura de segurança mais conservadora.
- **Sem lista de sessões ativas visível ao usuário:** o modelo permite revogar tudo (troca de senha, logout) mas não expõe "você está logado em 3 dispositivos, deseja encerrar um específico?" — funcionalidade que o schema (refresh_tokens por user_id) já suportaria, mas que não foi construída.

Como este diagrama deve evoluir

Se uma funcionalidade de "gerenciar sessões ativas" for construída, este diagrama precisa ganhar um quarto bloco mostrando a listagem/revogação seletiva de refresh_tokens por usuário. Se o tempo de vida do access token ou refresh token mudar (hoje configurável via ACCESS_TOKEN_EXPIRES_IN / REFRESH_TOKEN_EXPIRES_IN), atualize os valores citados aqui e em system-overview.md .